Use of AI assistants. Problems 2.1 to 2.4 are No AI. Problem 2.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Racket file**, built from the template: code as code, traces and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper, whichever problem it comes from.

Problem 2.1 (Explicit recursion). No AI Write the three functions below by explicit recursion: no `map`, `filter`, `foldl`, `apply`, `andmap`, `ormap`, and no **`for`** loops. For each, say whether your definition is tail recursive, and why you made it so or not.

(a) `balances`: an account starts at zero, and the answer is the balance after each transaction.

```
(balances '(300 -120 50)) ; '(300 180 230)
(balances '())            ; '()
```

(b) `interleave`: the elements of two lists in turn, stopping as soon as either list runs out.

```
(interleave '(a b c) '(1 2 3)) ; '(a 1 b 2 c 3)
(interleave '(a b) '(1 2 3))   ; '(a 1 b 2)
```

(c) `low-high`: the lowest and the highest of a non-empty list of readings, as a pair, in **one pass** over the list.

```
(low-high '(7 2 9 4)) ; '(2 . 9)
(low-high '(5))       ; '(5 . 5)
```

Problem 2.2 (Tail calls). No AI A sensor reports a reading every minute. The function counts how many times a reading came out lower than the one before it:

```
(define (count-drops readings)
  (cond
    [(empty? readings) 0]
    [(empty? (rest readings)) 0]
    [(< (second readings) (first readings))
     (+ 1 (count-drops (rest readings)))]
    [else (count-drops (rest readings))]))
```

(a) There are two recursive calls. For each, say whether it is a tail call, and why.

(b) Trace `(count-drops '(5 3 4 1))` in the substitution model, one step per line, and mark the largest expression that appears.

(c) Rewrite it as `count-drops2`, with an accumulator, so that every recursive call is a tail call, and trace the same input.

Problem 2.3 (A proof for every list). No AI Two functions by explicit recursion:

```
(define (my-length lst)
  (cond [(empty? lst) 0]
        [else (+ 1 (my-length (rest lst)))]))

(define (my-append xs ys)
  (cond [(empty? xs) ys]
        [else (cons (first xs) (my-append (rest xs) ys))]))
```

(a) Check by two traces that these give the same number:

```
(my-length (my-append '(a b) '(c)))
(+ (my-length '(a b)) (my-length '(c)))
```

(b) Prove that, for *every* pair of lists `xs` and `ys`,

```
(my-length (my-append xs ys)) = (+ (my-length xs) (my-length ys))
```

in the style of the lecture: a list is `empty` or `(cons z zs)`, so there are two cases, and in the second you may assume the claim for the shorter list `zs`. You may use that taking `first` or `rest` of `(cons z zs)` gives `z` or `zs`.

(c) In one sentence: which step of your proof used the assumption, and why is assuming the claim for `zs` allowed?

Problem 2.4 (A counterexample). No AI A shop gives 10% off any amount of 100 or more. Two ways to total a cart, given `my-sum` from the lecture:

```
(define (discount x)
  (cond [(>= x 100) (* x 9/10)] [else x]))

(define (checkout cart)
  (discount (my-sum cart)))

(define (checkout2 cart)
  (cond [(empty? cart) 0]
        [else (+ (discount (first cart)) (checkout2 (rest cart)))]))
```

(a) Show that the two agree on every cart of *one* item, without trying values: trace both of these for an unknown price `p`.

```
(checkout (list p))
(checkout2 (list p))
```

(b) Give a cart of two items on which they disagree, and trace both on it.

(c) Replace `discount` by `(lambda (x) (* x 9/10))`, no threshold. In one or two sentences, why do the two then agree on every cart? Name the property of arithmetic you rely on.

Problem 2.5 (An accumulator, audited). A conversion, by naive recursion:

```
(define (to-fahrenheit temps)
  (cond [(empty? temps) '()]
        [else (cons (+ 32 (* 9/5 (first temps)))
                    (to-fahrenheit (rest temps)))]))
```

```
(to-fahrenheit '(0 100 -40)) ; '(32 212 -40)
```

Its recursive call is not a tail call, so a long list leaves a long chain of pending `cons` applications.

(a) AI allowed Ask an assistant to rewrite it with a tail-recursive helper and an accumulator parameter, and record what it produces. Any follow-up you put to it belongs here too, and in the transcript. Paste the definition above, and then a prompt along these lines:

> Rewrite `to-fahrenheit` in Racket with a tail-recursive helper function and an accumulator parameter.

Use a **basic, free** model, not the strongest you can reach: a strong one may just get it right.

(b) No AI Evaluate its answer on `'(0 100 -40)` by hand, in the substitution model, and say whether it agrees with the original. If it does not, say what went wrong, in the language of the model: which element ends up where, and why. If it does, read what it changed and say what avoided the trap. Either way the evidence is the code it wrote, **not** its account of what it did.

(c) No AI Give a correct tail-recursive version, as `to-fahrenheit/acc`. There are two ways to make one, and they differ in cost. Give one, and say what it costs on a list of n readings.

Use of AI assistants. Part (a) only. Alongside your answer file, submit `transcript.md`: the whole exchange, your prompts and the assistant's replies, verbatim and unedited. Parts (b) and (c) are answered in the answer file itself, in your own words, like every other problem on this sheet. You may **not** submit the assistant's answer as your own, and an answer to (b) saying *it agrees* with no trace shown scores nothing. If you use no assistant, do the rewrite yourself in (a) and answer (b) about your own first attempt.